

LangGraph ReAct Agent

with CLI Tools

Tutorial

LangGraph · LangChain · CLI Tools · Verifier Agent · Graph Routing

1

Goal of This System

Build a ReAct-style LLM agent using LangGraph + LangChain

Use CLI commands as executable tools (cat, printf, python3)

Add a verifier agent to validate tool outputs

Implement iterative reasoning: THOUGHT → ACTION → OBSERVATION

Create a structured graph-based autonomous agent

2 What Is ReAct?

ReAct = REASON + ACT

Reasons step-by-step

THOUGHT

Chooses the right tools

ACTION

Observes results

OBSERVATION

Continues planning

REASON AGAIN

3 Why LangGraph?

Traditional Loop

```
while True:  
    reason()  
    act()  
    observe()
```

LangGraph Graph

```
StateGraph(MessagesState)  
    → agent_node  
    → tools_node  
    → conditional_edges
```

Advantages of LangGraph:

✓ Modular nodes

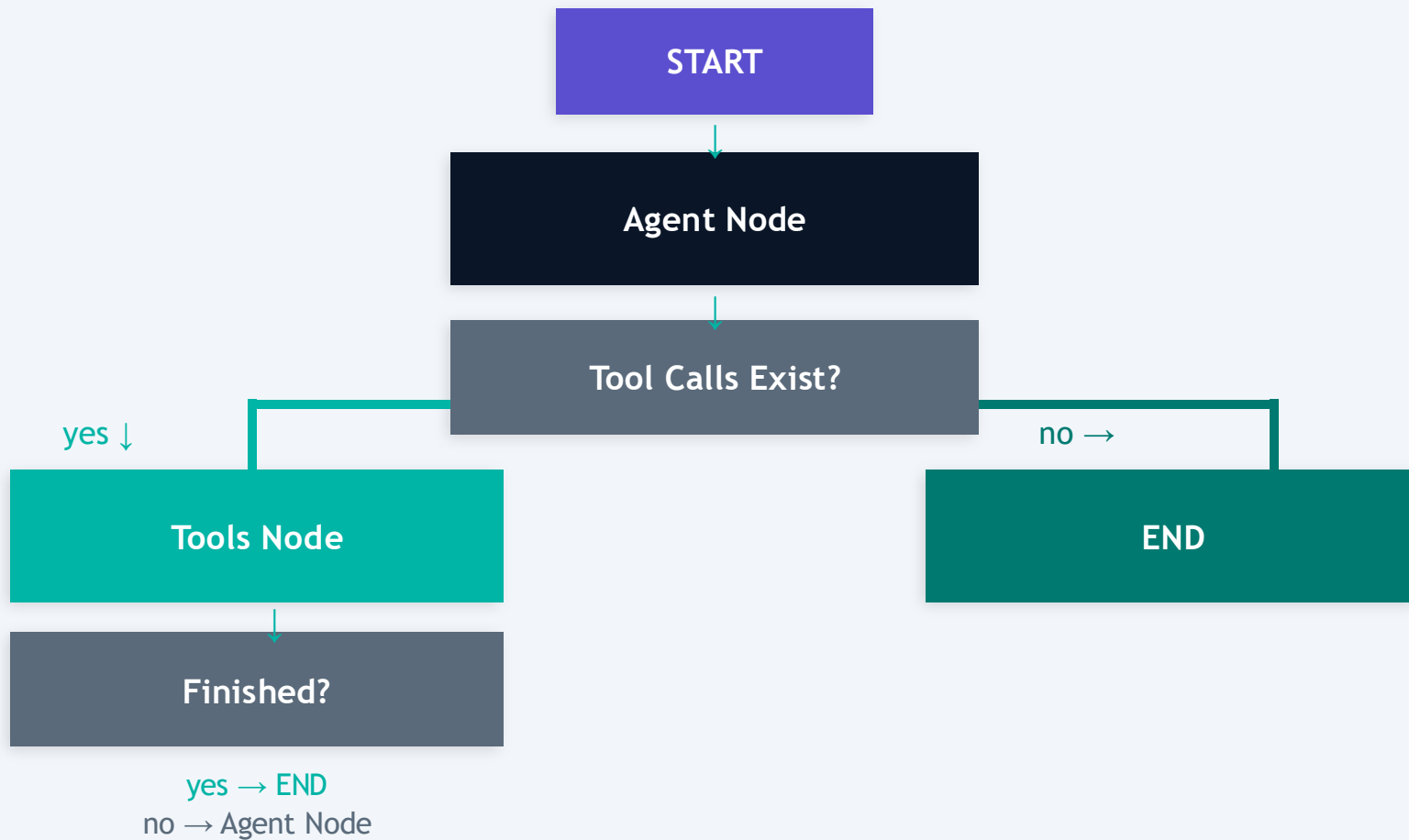
✓ Better debugging

✓ Multi-agent orchestration

✓ Explicit workflow

✓ Persistent memory

✓ Scalable architecture



5

Main Components



ReAct LLM Agent

Core reasoning engine powered by GPT-4o-mini



Tool Registry

TOOL_MAP mapping names → Python functions



LangGraph StateGraph

Graph-based workflow with nodes + edges



CLI Tool Layer

OS-level command execution via subprocess



Verifier Agent

Secondary LLM that validates tool outputs



Conditional Routing

Dynamic branching based on tool call results



Message Memory

MessagesState stores full conversation trace

LangGraph

Workflow graph — nodes, edges, conditional routing

LangChain

Tools, messages, ChatOpenAI, @tool decorator

OpenAI API

LLM backend — GPT-4o-mini inference

subprocess

CLI execution bridge — Python → OS commands

7 Core Idea of CLI Tools

The LLM itself CANNOT read files, run programs, or interact with the OS. CLI tools extend the model.



8 Tool 1 – Read File

```
@tool
def read_file(filename: str) -> str:
    """Read contents of a file using cat command."""
    result = subprocess.run(
        ["cat", filename],
        capture_output=True, text=True
    )
    return result.stdout
```

CLI Command: cat filename

Purpose: Inspect files · Read source code · Analyze configs

Output: stdout → returned as observation to LLM

9 Tool 2 – Write File

@tool

```
def write_file(filename: str, content: str) -> str:
    """Write content to a file using printf."""
    cmd = f'printf "%s" "{content}" > {filename}'
    result = subprocess.run(
        cmd, shell=True, capture_output=True, text=True
    )
    return "File written successfully"
```



Generate files



Patch code



Autonomous editing

Why printf Is Good

```
# OLD – echo  
echo "content" > file.txt  
  
# BETTER – printf  
printf "%s" "$content" > file.txt
```

Advantages over echo

- ✓ Preserves formatting perfectly
- ✓ Handles multiline content safely
- ✓ Safer shell escaping
- ✓ Production-quality shell usage

This is a production-quality improvement over the naive echo approach.

11 Tool 3 – Execute Python

```
@tool
def execute_python(script_path: str) -> str:
    """Execute a Python script and return output."""
    result = subprocess.run(
        ["python3", script_path],
        capture_output=True, text=True
    )
    return result.stdout or result.stderr
```



Generate Code



Execute Code



Observe Result

= Foundation of Coding Agents

12 Tool 4 – Finish

```
@tool
def finish(answer: str) -> str:
    """Explicitly terminate the workflow and provide final answer."""
    return f'FINISHED: {answer}'
```

Explicitly terminates the LangGraph workflow

Provides the final answer to the user

Critical — without it, the agent may loop forever

```
# All tools in one list
TOOLS = [read_file, write_file, execute_python, finish]

# Dynamic name → function mapping
TOOL_MAP = {t.name: t for t in TOOLS}

# Usage
fn = TOOL_MAP[tool_name]
result = fn.invoke(args)
```



Dynamic tool lookup by name



Enables dispatch at runtime



Easy to extend — just add to TOOLS list

14 Binding Tools to the LLM

```
# Critical line – exposes tools to the model
llm = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0
).bind_tools(TOOLS)

# The model now auto-generates structured tool calls:
# { "name": "read_file", "arguments": {"filename": "hello.txt"} }
```

WHICH tool

WHEN to call it

WHAT arguments

The model automatically decides – no manual parsing needed

15 System Prompt

```
SYSTEM_PROMPT = '''  
You are a ReAct agent. Think step-by-step.  
  
For each task:  
    1. THINK about what you need to do  
    2. USE a tool to take action  
    3. OBSERVE the result  
    4. REPEAT or call finish() when done  
  
Always call finish() to terminate.  
'''
```

Teaches the model to think step-by-step

Instructs when and how to use tools

Without this prompt — model behaves like a chatbot

16 LangGraph State

```
from langgraph.graph import MessagesState

# MessagesState automatically manages:
class MessagesState(TypedDict):
    messages: Annotated[list, add_messages]

# Contains: HumanMessage | AIMessage | ToolMessage
```

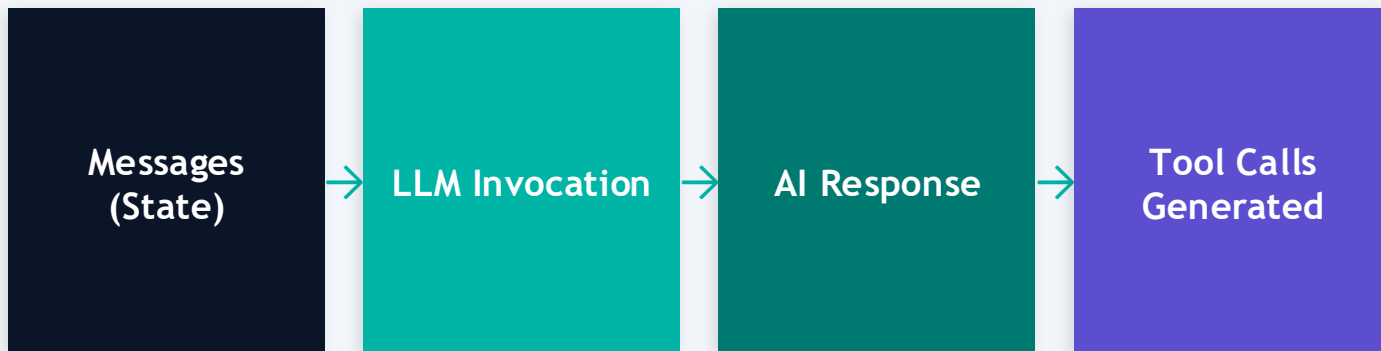
HumanMessage → User input / initial query

AIMessage → LLM reasoning + tool call proposals

ToolMessage → Tool execution results / observations

17 Agent Node

```
def agent_node(state: MessagesState):  
    """Run the LLM to generate reasoning and tool calls."""  
    messages = state['messages']  
    # Prepend system prompt  
    full_messages = [SystemMessage(SYSTEM_PROMPT)] + messages  
    response = llm.invoke(full_messages)  
    return {'messages': [response]}
```



```
def tools_node(state: MessagesState):  
    """Execute tool calls and collect observations."""  
    tool_messages = []  
    for tool_call in state['messages'][-1].tool_calls:  
        tool_fn = TOOL_MAP[tool_call['name']]  
        result = tool_fn.invoke(tool_call['args'])  
        # Optionally verify result  
        verified = verify_tool_result(tool_call['name'], result)  
        tool_messages.append(ToolMessage(verified, tool_call_id=tool_call['id']))  
    return {'messages': tool_messages}
```

Tool Call



Python Fn



CLI Command



Result



ToolMessage

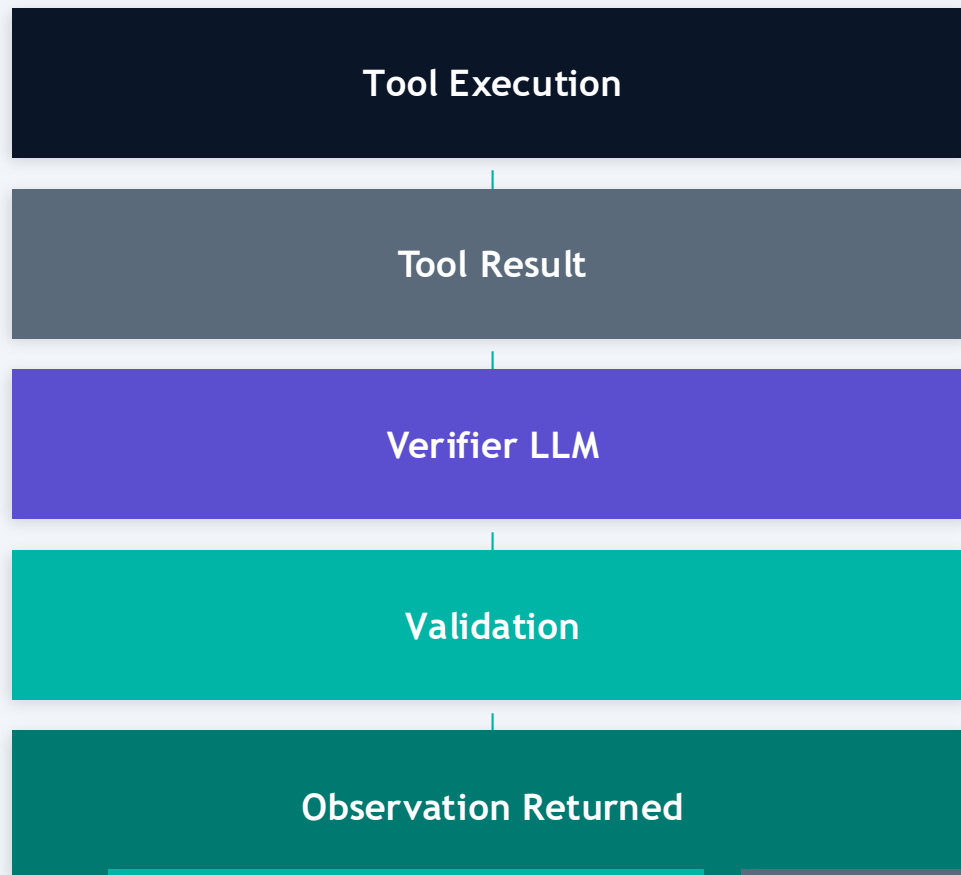
19 Verifier Agent

```
def verify_tool_result(tool_name: str, args: dict, result: str) -> str:
    """Secondary LLM that validates tool output."""
    prompt = f"""
    Tool: {tool_name}, Args: {args}
    Result: {result}
    Is this output correct? Reply OK: or ISSUE: """
    verdict = verifier_llm.invoke(prompt).content
    return verdict + '\n' + result
```

ReAct Agent → Reasoning + Planning

CLI Executor → Command Execution

Verifier Agent → Correctness Checking



Possible verdicts:

OK: looks correct

ISSUE: suspiciously empty

21 Why Verifier Matters

Without Verifier

Tool failures may silently propagate.
Agent continues on bad assumptions.

With Verifier

Agent detects suspicious outputs immediately.
Can recover, retry, or escalate.

Catches:

 Empty file output

 Runtime errors

 Missing content

 Malformed
execution

22 Conditional Routing in LangGraph

The most important LangGraph concept — dynamic branching based on state.

```
# Routing Function 1
def should_continue(state):
    last = state['messages'][-1]
    if last.tool_calls:
        return "tools"
    return END
```

```
# Routing Function 2
def after_tools(state):
    last = state['messages'][-1]
    if "FINISHED" in last.content:
        return END
    return "agent"
```

```
# Register conditional edges
graph.add_conditional_edges("agent", should_continue, {"tools": "tools", END:
END})
graph.add_conditional_edges("tools", after_tools, {"agent": "agent", END:
END})
```

Why Conditional Edges Are Powerful

Creates Dynamic Execution Flow — not a fixed pipeline



Loop

Return to agent after tool use



Branch

Take different paths by state



Retry

Re-run steps on failure



Terminate

Exit dynamically at any point

```
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import MessagesState

# 1. Create graph with state schema
graph = StateGraph(MessagesState)

# 2. Register nodes
graph.add_node("agent", agent_node)
graph.add_node("tools", tools_node)

# 3. Add fixed edges
graph.add_edge(START, "agent")

# 4. Add conditional routing
graph.add_conditional_edges("agent", should_continue)
graph.add_conditional_edges("tools", after_tools)
```

```
# Compile: converts definition → executable runtime  
app = graph.compile()
```



The compiled app is a fully executable LangGraph runtime engine.

26 Running the Workflow

```
# Launch the graph execution engine
result = app.invoke({
    'messages': [
        HumanMessage(content=
            "Read hello.txt, uppercase it, write to upper.txt, execute test.py"
        )
    ]
})

print(result['messages'][-1].content)
```

What happens next:

→ agent_node runs the LLM → generates tool calls → tools_node executes them
→ verifier validates → loop continues until finish() is called → END

27 Example Full Workflow

Step 1 — Agent Node

LLM reasons → calls `read_file("hello.txt")`

Step 2 — Tools Node

CLI: `cat hello.txt` → "hello world" → verifier OK

Step 3 — Agent Node

LLM reasons → calls `write_file("upper.txt", "HELLO WORLD")`

Step 4 — Execute Python

CLI: `python3 test.py` → result observed

Step 5 — Finish

LLM calls `finish("Task completed")` → END

28 Why This Is Better Than Simple Loops

```
# Traditional
while True:
    reason(); act(); observe()
    if done: break
```

LangGraph:
Structured Graph-based Workflow

Explicit graph	Easier debugging & visualization
Modular nodes	Reusable, testable components
Routing	Dynamic workflows, no hardcoding
Memory	Persistent state across steps
Recursion limits	Safer execution, no infinite loops
Multi-agent support	Scalable to complex systems

Similarity to Modern Agent Systems



Anthropic Claude Code



OpenAI Coding Agents



Cursor



OpenHands

Core Pattern:

LLM

+ Tools

+ Verification

+ Iterative
Graph

+ Memory

The LLM NEVER directly executes commands.

LLM proposes action

Runtime executes safely

Observation returned

LLM continues reasoning



Safety



Logging



Monitoring



Permissions



Debugging

31 Safety Problems



Arbitrary command execution — any shell command can run



Unrestricted filesystem access — can read/write anywhere



Malicious shell usage — e.g. `rm -rf /` could be triggered

```
# Example dangerous query an agent might generate:  
>>> execute_python("cleanup.py") # script contains: import os; os.system("rm -rf /")  
  
# Or direct shell injection via write_file content
```

32 Recommended Safety Improvements

```
# A. Permission Layer
def safe_execute(cmd):
    print(f'Allow: {cmd}? [y/n]')
    if input() == 'y':
        return subprocess.run(cmd)

# B. Command Whitelist
SAFE_COMMANDS = ['cat', 'printf',
                 'python3']
assert cmd[0] in SAFE_COMMANDS
```

✓ User Permission Layer

✓ Command Whitelist

```
# C. Docker Sandbox
subprocess.run([
    'docker', 'run', '--rm',
    '-v', '/workspace:/workspace',
    'sandbox-image',
    'python3', script
])

# D. Restrict to /workspace/ only
```

✓ Docker / VM / Firejail Sandbox

✓ Restricted Workspace (/workspace/)

33 Major Educational Concepts Demonstrated

✓ ReAct Agents

✓ CLI Tools

✓ Tool Calling

✓ Graph Routing

✓ Autonomous Coding Agents

✓ LangGraph Workflows

✓ subprocess Execution

✓ Verifier Agents

✓ State Management

✓ Multi-Agent Workflows

Recommended Future Extensions



Planner node



Repair node



Retry logic



Long-term memory



Vector database



Git integration



Docker tool



Bash tool



Web search tool



MCP server



Multi-file editing



Auto debugging